

Session 3 — Foundations 3

Developer tools (Git, VS Code, Python, containers, Jupyter)

Qiuyu Yu, Human Development and Family Science (HDFS), Auburn University

Recap: Sessions 1–2 gave you the Linux command line and got you onto the HPC. You can submit and monitor a job.

Goal today: set up the tools you'll use every day: version control (Git/GitHub), an editor (VS Code), Python environments, containers, and Jupyter.

Roadmap for today

1. **Version control** (Git & GitHub)
2. **VS Code** (your editor / IDE, and editing on the HPC with Remote-SSH)
3. **Python environments** (venv, conda, uv) and containers
4. **Jupyter Notebook**

This is Part 3 of the Foundations. Next session we start on the data: how the scanner makes it, and the preprocessing pipeline.

1. Version control: Git & GitHub

- **Git** = a tool that records every change to your files, so you can see the history and roll back.
- **GitHub** = a website that hosts your Git project online, for backup, sharing, and collaboration.
- A **repository** ("repo") = a project folder that Git is tracking.
- Why bother: never lose work, see exactly what changed and when, and let others reproduce you.
- Best for **scripts and code** (version control shines on text). You can keep other files too, but GitHub has **storage limits**, so large data and outputs don't belong there.

1a. First-time Git setup (once per machine)

Run these once in a **terminal** (your laptop's, or VS Code's integrated terminal, slide 2b). They tell Git who you are; this name and email label every commit you make.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main    # name the first branch "main"
git config --list                               # check what is set
```

- `--global` writes to `~/.gitconfig` and applies to all your repos on this machine.
- Use the **same email** as your **GitHub account** so commits link to your profile.
- For pushing, GitHub no longer takes a password: use an **SSH key** (Session 2) or a personal access token.

1b. Git: the everyday loop

```
git clone <url>      # get a repo onto this machine (first time only)
git status           # show what changed (run this constantly)
git add .            # stage your changes
git commit -m "message" # save a snapshot, with a note
git push             # upload snapshots to GitHub
git pull             # download the latest from GitHub
```

- Add a `.gitignore` so you commit **code only**, never data or large outputs.
- These are **terminal** commands; VS Code's Source Control buttons run the same loop (slide 2c).

1b2. Local vs remote: what the commands mean

Git saves versions **on your machine first**, then syncs to GitHub:

```
files --add--> staging --commit--> local history --push--> GitHub
```

- `git add` : mark which changes go in the next snapshot (the **staging area**).
- `git commit` : save a snapshot **locally**, with a message. Nothing leaves your computer yet.
- `git push` : upload your local commits to **GitHub** (the remote).
- `git pull` : download commits from GitHub and merge them into your local copy.
- **Sync** (the VS Code button) = pull, then push, in one click.

You can commit all day offline; only `push` / `pull` touch the network.

1c. Create a GitHub repo & link your project

On **github.com**: click **New** -> name the repo -> Public/Private -> **Create repository**.
Leave it empty (no README) if you already have local files.

Then link your existing folder and push:

```
cd my_project
git init                                # start tracking (if new)
git add . && git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:you/my_project.git
git push -u origin main                 # -u sets the upstream once
```

- `origin` is just a name for the remote URL; `-u` links your local `main` to it, so later you only type `git push`.
- Starting fresh instead? `git clone <url>` gives you the folder already linked.

1c2. Set up a fresh repo (LICENSE, README, .gitignore)

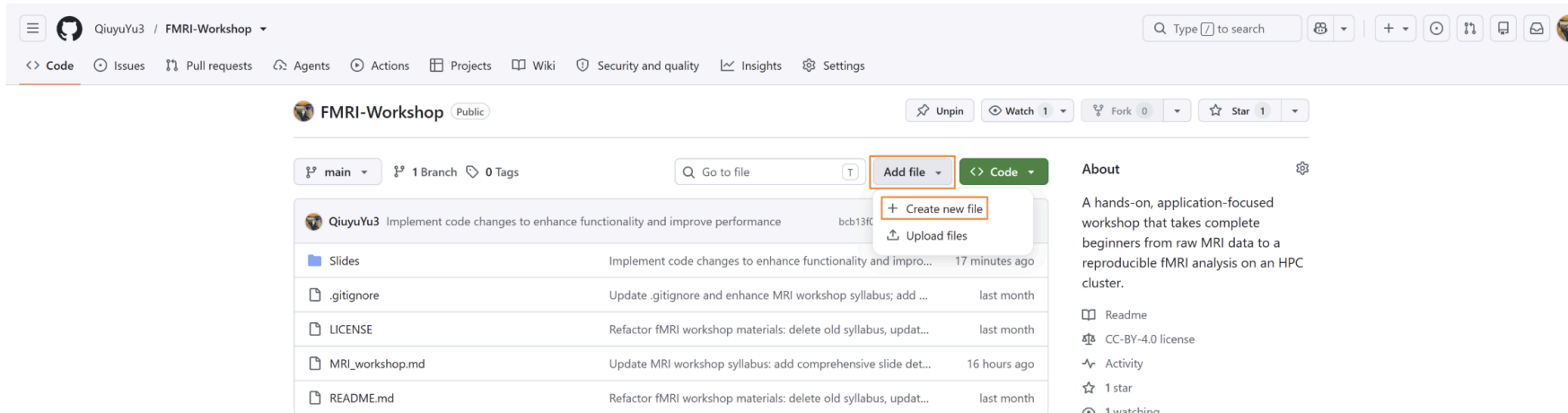
Three files worth adding to a new repo:

- **LICENSE**: says what others may legally do with your code. On GitHub, **Add file -> Create new file**, name it `LICENSE`, and a picker pops up; **MIT** is a common permissive choice.
- **README.md**: the repo's front page: what the project is and how to run it (GitHub shows it on the homepage).
- **.gitignore**: a list of files Git should **never track** (data, large outputs, env folders like `venv/`, `__pycache__/`). Keeps commits to **code only**.

Typical flow: add `LICENSE` on GitHub, add `README.md` + `.gitignore` locally, then `commit` and `push`.

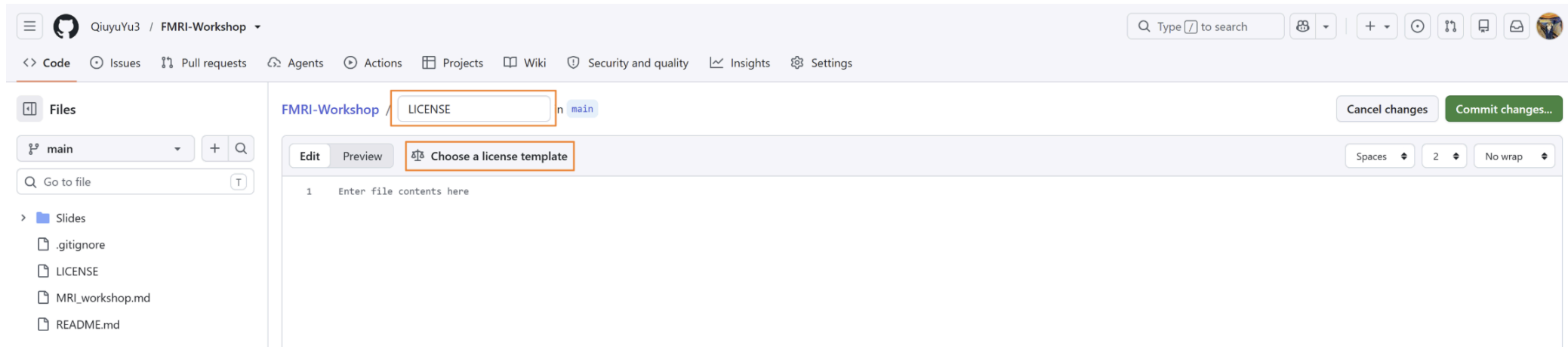
1c3. Adding a LICENSE on GitHub (1)

On the repo page: **Add file -> Create new file.**



1c4. Adding a LICENSE on GitHub (2)

Name the file `LICENSE` ; a **Choose a license template** button appears (MIT for a permissive default).



1d. Branches & merge

A **branch** is a parallel line of work, so you can try something without breaking `main`.

```
git switch -c feature      # create and switch to a new branch
# ... edit, add, commit as usual ...
git switch main           # back to the main line
git merge feature         # fold the branch's commits into main
```

- One branch per feature or experiment; keep `main` working.
- On GitHub you open a **pull request** to review a branch before merging.
- A **merge conflict**: two branches changed the same lines, so Git marks them and you choose what stays.
- Not just the terminal: **GitHub's website** and **VS Code** both do branches, pull requests, and merges through buttons.

1d2. Working with others

- Before you start, **pull** your collaborators' latest changes: `git pull` .
- If you both edited the **same lines**, Git can't auto-merge: it flags a **conflict**, and you **merge by hand** (keep yours, theirs, or a mix), then commit.
- Habit: pull often, commit small, push often, so conflicts stay small.

1e. Undo & inspect history

Everyday safety net, before you panic:

```
git status          # what changed, what is staged
git log --oneline    # history of commits
git diff            # exact line changes, not yet staged
git restore file.py  # discard un-committed changes to a file
git restore --staged f # unstage (undo a git add), keep the edits
git commit --amend    # fix the last commit (message or contents)
```

- Committed work is safe: almost anything can be recovered, so commit often.

2. VS Code: your editor / IDE

- An **IDE** = a text editor with extras: highlighting, an integrated terminal, git buttons, extensions.
- Why VS Code: free, beginner-friendly, great Python / Jupyter support.
- Open a folder: *File -> Open Folder*, or type `code .` in a terminal.
- Key extensions: Python, Jupyter, and **Remote-SSH** (next slide).

2a. Edit files on the HPC with Remote-SSH

No more nano/vim for everyday editing; work on cluster files in the full GUI:

1. Install the **Remote - SSH** extension in your local VS Code.
2. Press F1 -> **Remote-SSH: Connect to Host** -> `user@cluster` (passwordless with your SSH key).
3. **File** -> **Open Folder** -> your project on the cluster.
 - The integrated terminal now runs **on the cluster**, so you still `sbatch` from there.
 - It runs on the **login node**, fine for editing; keep heavy compute off it.
 - `nano` / `vim` stay handy for quick fixes, or when there is no server.

2a2. Remote-SSH is a workload, not just an editor

VS Code on the cluster is not lightweight. Remote-SSH starts a **server on the login node** (`code-server` , `node` , an extension host, file watchers) and keeps it running.

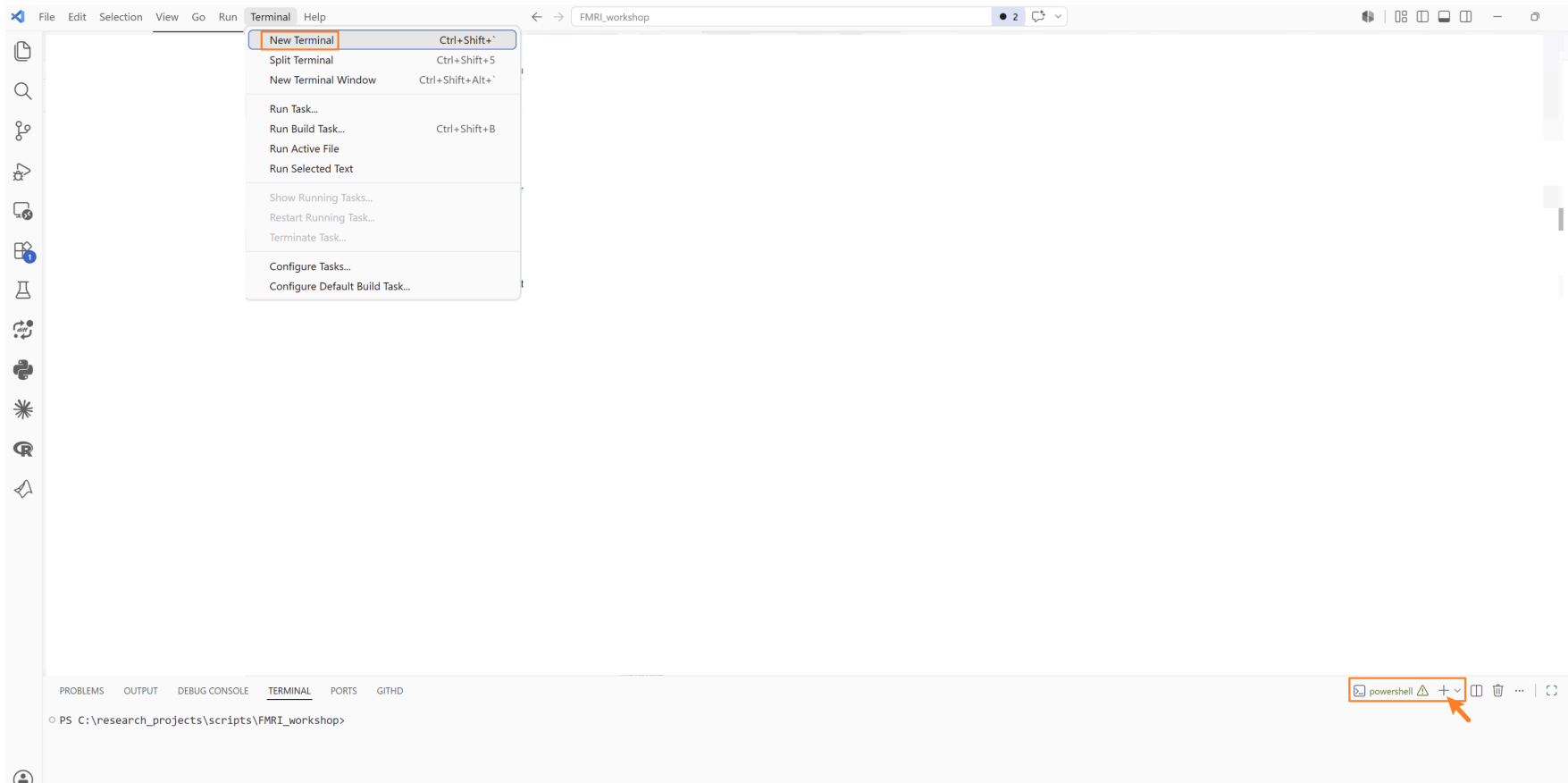
- **Extensions auto-run in the background:** Python, Pylance, Jupyter, Copilot, Docker burn CPU / RAM on the login node even while you sit idle. **Disable the heavy ones on the remote**, and turn them on only inside an interactive job.
- **Editing on the login node is fine; running code is not.** To actually run things, start an **interactive job** first, or use a VS Code **tunnel** launched inside a job so every process sits on a compute node.
- `~/ .vscode-server/` grows in your **home directory**, which has a tiny quota (Session 2). Clean it out if home fills up.

2b. The integrated terminal

1. Open it: **Terminal -> New Terminal**, or **Ctrl+`** (backtick). It opens in your project folder.
 2. **Switch shells:** the dropdown next to  picks a profile. On Windows the default is **PowerShell**; choose **Git Bash** or **WSL** to get a real bash.
 3. Set the default: **Terminal -> Select Default Profile**.
- **Git Bash** only appears after you install **Git for Windows** (it ships with it); **WSL** needs WSL installed.

2b2. The integrated terminal: New Terminal & shell picker

Terminal -> New Terminal; the shell dropdown (bottom-right of the panel) switches profile.



2b3. The integrated terminal (cont.)

- **Split panes:** run an editor, `squeue`, and a log `tail -f` side by side.
- On **Remote-SSH** it is the cluster's login shell (bash), so `module load` and `sbatch` behave exactly as in a normal SSH session.
- It inherits that session's environment: variables, `PATH`, and loaded modules.

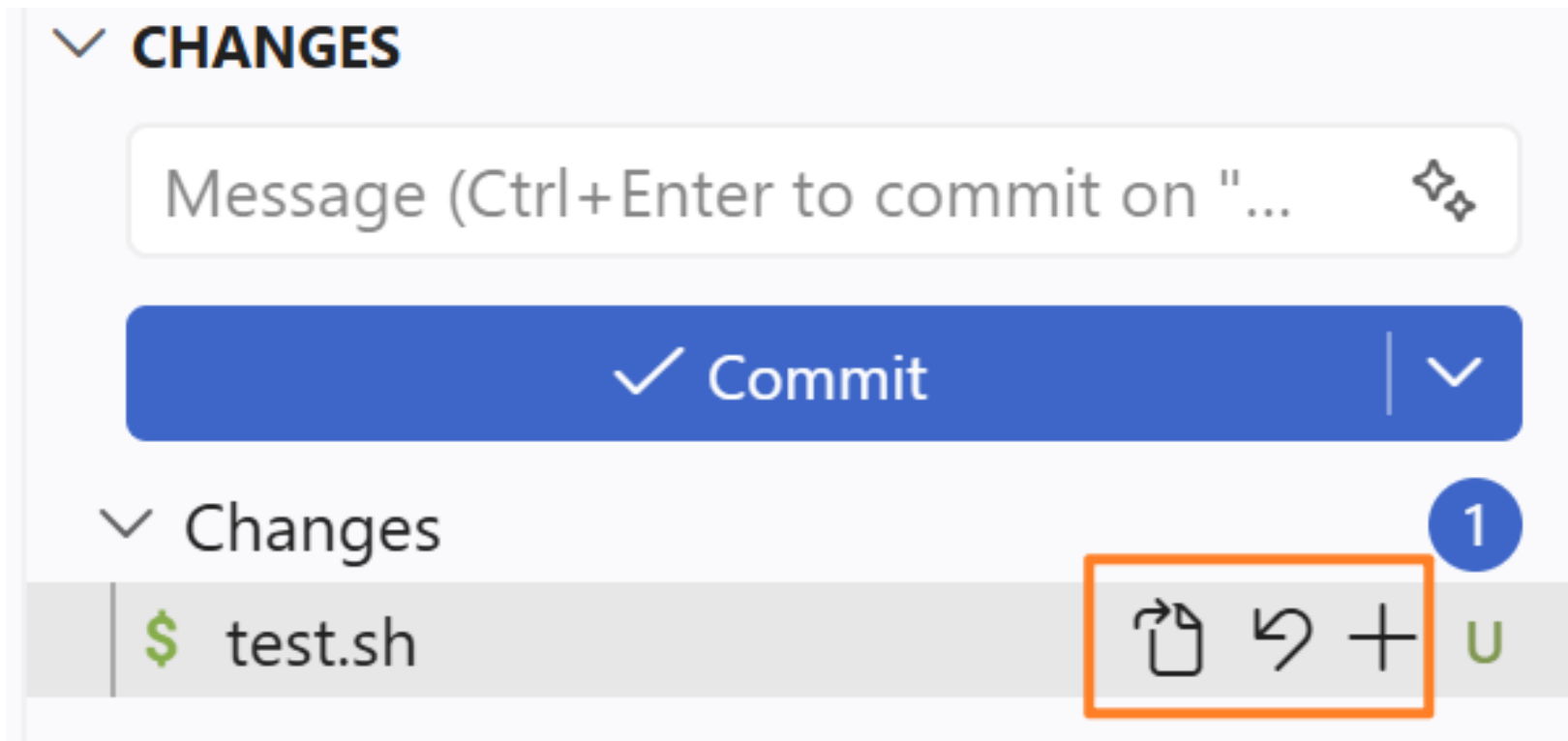
2c. Git & GitHub inside VS Code

Once you have signed in, VS Code drives the whole loop with buttons instead of commands.

1. **Sign in:** Accounts icon (bottom-left) -> **Sign in with GitHub**.
 2. **Get a repo:** Command Palette (Ctrl+Shift+P) -> **Git: Clone** -> pick your repo -> open the folder.
 3. **Or publish a local folder:** Source Control panel (Ctrl+Shift+G) -> **Publish to GitHub**, which creates the repo for you.
- Daily use in the **Source Control** panel: `+` stages a file, type a message, click **Commit**, then **Sync** to push/pull.
 - Those buttons are exactly `git add` / `git commit` / `git push`; the terminal still works alongside them.

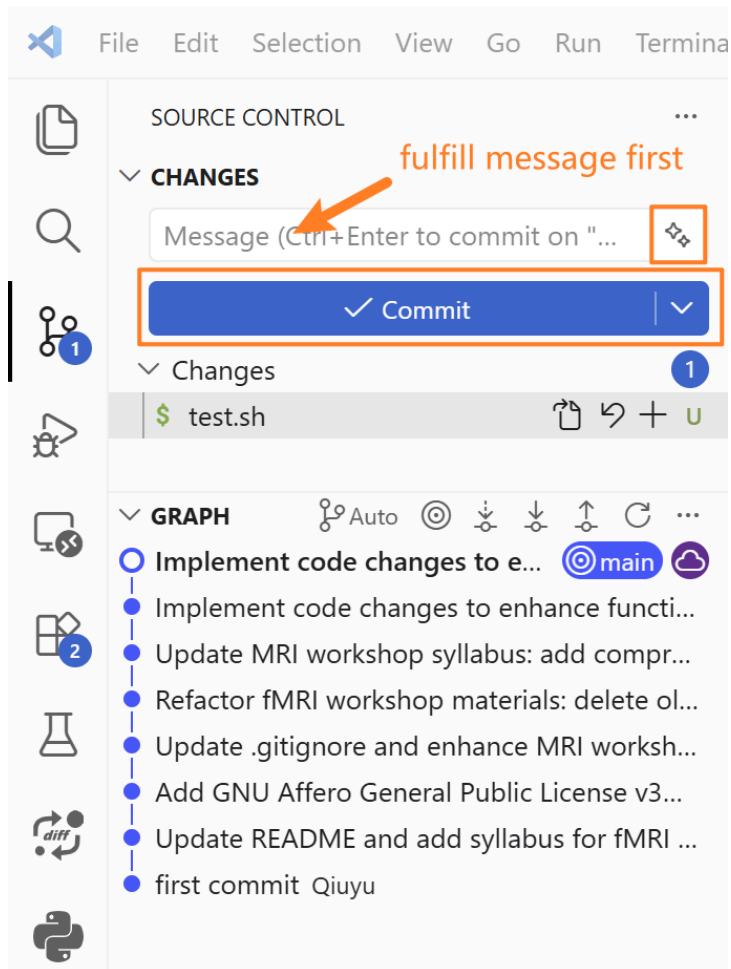
2c2. Source Control panel: a file's actions

On each changed file, hover to get: open · discard changes (undo) · stage (+).



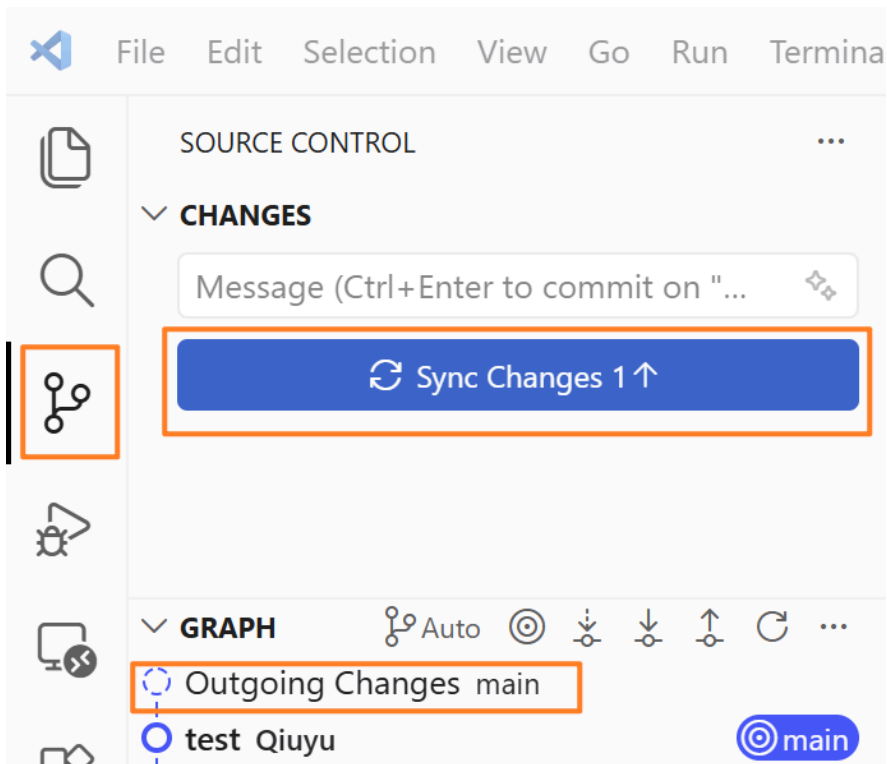
2c3. Write a message & commit

Fill in the **message** first, then click **Commit**. This saves the snapshot **locally**.



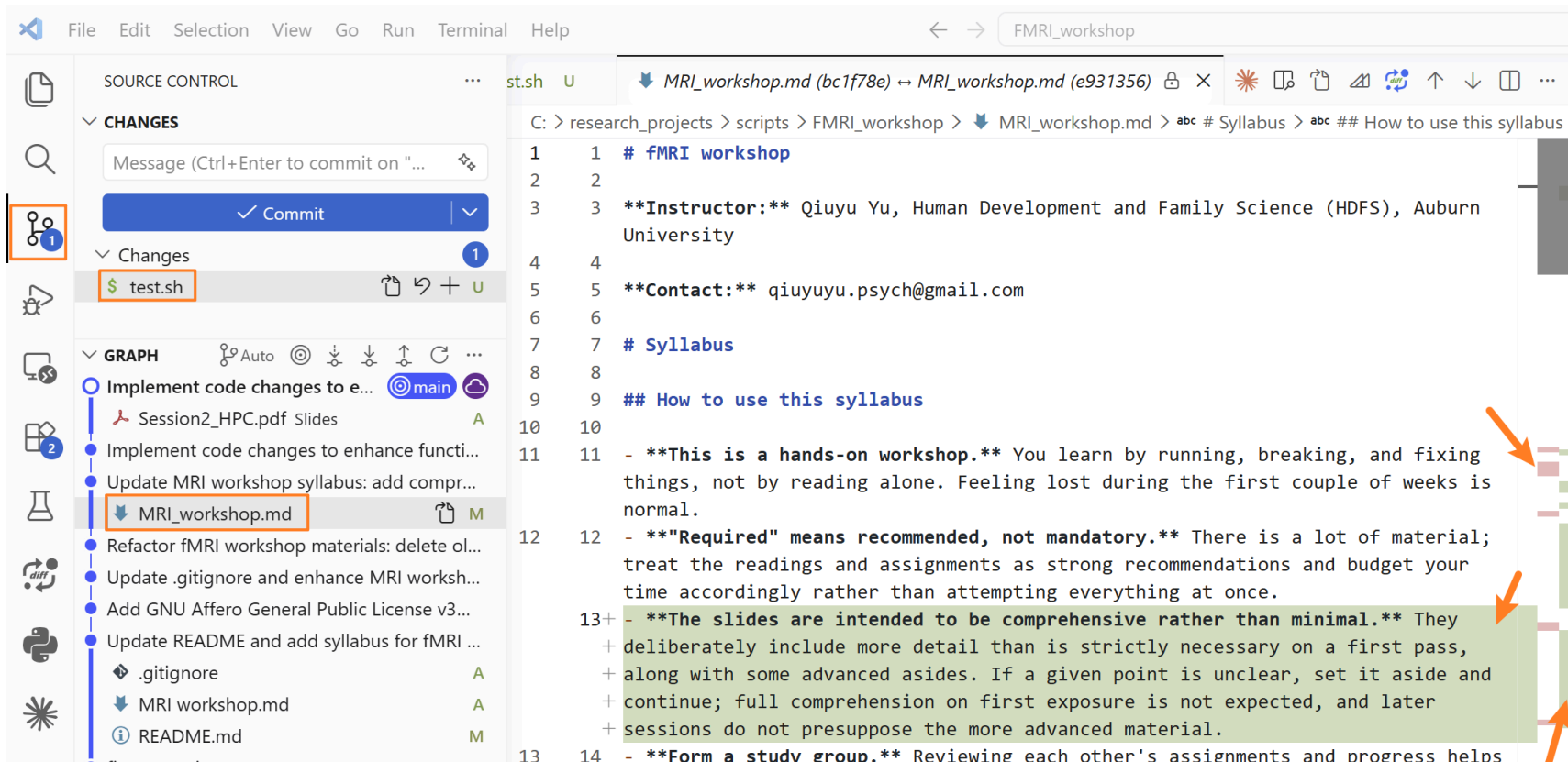
2c4. Commit is local: Sync to push

- After **Commit**, the change is still **local** (the graph shows **Outgoing Changes**).
- Click **Sync Changes** to push it to GitHub. Sync also pulls.



2c5. See diffs & history

- Click a changed file to see its **diff** (green = added, red = removed).
- The **GRAPH** shows commit history; click a commit to see its changed files, and follow a file's **modification history**.



2d. Useful VS Code extensions

- **Remote:** Remote - SSH / Remote Development (edit on the HPC); WSL (run Linux on Windows locally)
- **Notebooks:** Jupyter
- **Git:** Git History / Git Graph (see history visually); .gitignore Generator
- **Data files:** Rainbow CSV, reads BIDS `.tsv`, AFNI `.1D`, QC tables as colored columns
- **Writing:** Markdown Preview Enhanced; LaTeX Workshop (papers)
- **(Optional) ShellCheck:** lint bash scripts (VS Code's built-in is highlighting only, not bug-checking)

3. Python environments: three ways

```
# 1) venv + pip : built into Python, lightweight (Python packages only)
python -m venv fmri && source fmri/bin/activate
pip install nibabel nilearn
```

```
# 2) conda : heavier, also manages the Python version + compiled/system libs
conda create -n fmri python=3.11 && conda activate fmri
```

```
# 3) uv : new, very fast drop-in for pip + venv
uv venv && uv pip install nibabel nilearn
```

3a. venv, conda, uv: which to use

- **venv / uv**: light, Python-only; you supply Python (e.g. `module load python`).
- **conda**: when you need a specific Python version or non-Python dependencies, with no admin rights.
- The difference is how much each one manages: just packages, or Python plus system libraries too.

3a2. Pin & manage your environment

- Pin it so others (and future you) can rebuild the exact setup:

```
pip freeze > requirements.txt      # venv / pip  
conda env export > environment.yml # conda
```

- Rebuild from it: `pip install -r requirements.txt` or `conda env create -f environment.yml`.
- **On HPC:** conda environments are large (many files). Home is often fine, but if your **home quota** is small, put them in work space instead. Check your site's policy.

3b. Why environments & containers exist

- Different tools need different, often **conflicting**, package versions ("dependency hell").
- A **conda environment** isolates one project's packages from another's.
- A **container** (Singularity/Apptainer) freezes a whole software stack, OS libraries included, so it runs the same everywhere and stays reproducible.
- On HPC you use **Singularity / Apptainer**, not Docker: Docker needs root, which shared clusters don't give you.
- Rule of thumb: conda for your own Python; containers for big fixed pipelines (fMRIPrep, MRIQC).

4. Jupyter Notebook

An interactive notebook: write code and see its output (text, tables, plots) inline.

```
jupyter notebook          # opens in a browser
```

- Or launch it from your HPC's web portal as an "interactive app".
- Or open the `.ipynb` **right inside VS Code** (Jupyter extension): same kernels, run cells inline, no browser.
- A notebook is a list of **cells**: code, or Markdown for notes.
- Run a cell with **Shift+Enter**; the **kernel** keeps your variables in memory.
- Pick the kernel that matches your conda environment.
- Best for: exploring data, quick plots, and prototyping before you write a script.

4b. Jupyter: kernels & version control

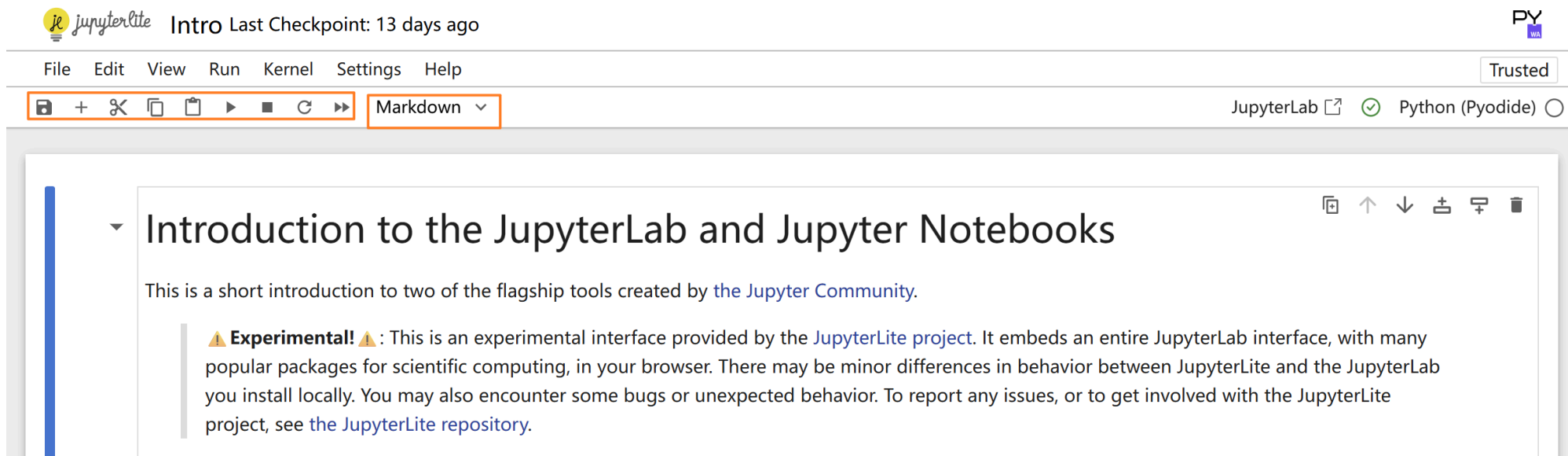
- Make your env show up as a kernel (otherwise Jupyter can't see it):

```
python -m ipykernel install --user --name fmri --display-name "Python (fmri)"
```

- Then pick "Python (fmri)" as the notebook's kernel.
- **Notebooks and Git:** `.ipynb` stores outputs too, so diffs are messy. `nbstripout` strips outputs so Git tracks only the code.

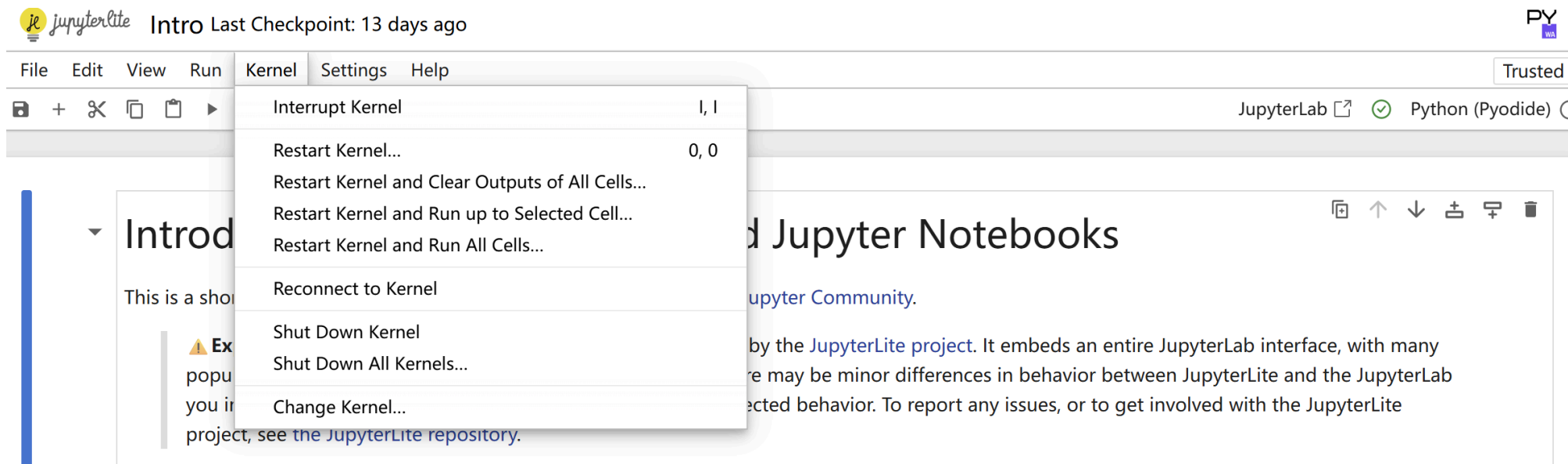
4c. The Jupyter interface: toolbar & cells

- **Toolbar:** save, add a cell, run ►, stop, restart ↺, run-all.
- The dropdown sets each cell to **Code** or **Markdown**; the active **kernel** shows top-right.



4c2. The Jupyter interface: the Kernel menu

interrupt a stuck cell, **restart** (clears every variable in memory), or **Change Kernel** to switch environments.



4d. Jupyter on the HPC (a bit more involved)

Two extra steps versus your laptop:

1. **Launch it in the HPC portal** (Open OnDemand) as an interactive app, or tunnel it over SSH. It runs as a **job on a compute node** (Session 2).
 2. **Register your env as a kernel** (slide 4b), but first install the bridge **inside that env**:

```
pip install ipykernel
```
- **Match versions:** pick a portal Jupyter whose **Python / toolchain** matches your environment, or the kernel misbehaves.
 - Kernel missing from the list? `ipykernel` was not installed **inside that env**.

Assignment (Session 3)

1. Set your Git identity (`git config --global user.name / user.email`)
2. Create a GitHub repo and push one script (from the terminal, or via VS Code)
3. Connect VS Code to the HPC with **Remote-SSH** and open your project folder
4. Create a Python environment (venv or conda) and install `nibabel` / `nilearn`
5. Create and run a Jupyter Notebook (via your HPC's portal), using that environment's kernel

Reading: <https://qiuyuyu3.github.io/fMRI-Data-Processing-Manual-on-HPC/introduction/>

You can now...

- track your work with Git and push it to GitHub
- edit files on the HPC from VS Code with Remote-SSH
- create an isolated Python environment, and know when to reach for a container
- run a Jupyter Notebook against the right kernel

Next: how the scanner makes data, and the preprocessing pipeline.